

SchedPilot 项目计划书

面向云原生高性能负载的 eBPF/sched_ext 自适应调度系统

基于 BPF/sched_ext 的用户态高性能自适应调度方向

项目版本: v0.2 编制日期: 2026-08-30

项目计划书初稿

目录

一、方案摘要

二、赛题理解与需求响应

三、需求分析与目标场景

3.1 目标问题

3.2 首验证场景

3.3 应用场景

四、总体解决方案

4.1 用户态控制面

4.2 eBPF 观测面

4.3 特征与分类引擎

五、核心创新与技术优势

5.1 面向业务 SLO 的闭环调度

5.2 CPU/缓存/内存访问模式联合分类

5.3 CPU-LLC-NUMA 多级拓扑感知

5.4 用户态快速演进、内核态低开销执行

5.5 可解释、可回滚、可审计

5.6 与现有方案的差异

六、关键技术与调度算法

6.1 多级 DSQ 设计

6.2 策略动作

6.3 动态门控与稳定性

6.4 Loader 与 Watchdog

七、实施计划

7.1 报名与校赛阶段：2026-08-30 至 2026-09-25 12:00

7.2 省赛强化阶段：2026-09-26 至 2026-10-31

7.3 总决赛完善阶段：2026-11-01 至 2026-11-30 11

八、测试与性能验证方案 11

8.1 对照组设计 11

8.2 评价模式 11

8.3 负载矩阵 11

8.4 数据与统计要求 12

8.5 性能验收门槛 12

九、可行性、风险与安全机制 12

十、产业价值与推广方式 13

10.1 产业价值 13

10.2 推广路径 13

十一、项目成果与交付物 14

十二、团队分工建议 14

十三、v1.0 验收标准 14

十四、正式提交材料规划 15

一、方案摘要

云原生节点通常同时承载延迟敏感服务、吞吐型服务和后台计算任务。系统默认 fair 调度器能够提供通用公平性，但难以根据业务 SLO、任务运行行为、缓存局部性和内存访问特征进行主动调度；传统静态 CPU 配额或 cgroup 权重也难以随负载变化及时调整。

SchedPilot 面向 openEuler 24.03 LTS SP1+ 提供的可编程调度能力，并兼容上游 Linux 6.12+ sched_ext 接口，构建由用户态控制面、eBPF 运行时观测、特征分析引擎、策略引擎和内核态 sched_ext 调度数据面组成的自适应调度系统。

SchedPilot 的核心思想不是简单提高任务优先级，而是通过“任务行为感知-在线分类-拓扑感知调度-性能反馈”闭环，让调度策略随业务状态动态变化。项目首个核心验证场景为裸机/cgroup 环境下的 Redis 延迟敏感服务与 CPU/内存密集型后台任务混部；在完成可归因的性能闭环后，再扩展到容器和 Kubernetes 部署。

项目目标是在同等硬件、同等内核、同等负载和同等资源约束下，相比系统默认 fair 调度器，在 Redis 混部场景中实现吞吐提升或 P99 延迟降低不低于 10%，并提供完整、可复现、可回滚的性能证据。

二、赛题理解与需求响应

赛题要求	SchedPilot 响应方案	验证方式
使用 BPF/sched_ext 机制实现用户态调度器	用户态 Agent 负责策略决策，eBPF 负责观测，sched_ext BPF scheduler 负责执行	scheduler 状态、BPF map、DSQ dispatch、运行计数
感知任务 CPU/内存访问模式	采集 runtime、wakeup、IPC、LLC miss、page fault、memory PSI、NUMA locality 等特征	特征快照、分类结果、缺失特征标记、离线回放
动态调整调度策略	在线调整任务类别、CPU placement、DSQ、时间片、迁移和抢占行为	策略切换日志、map 更新记录、调度统计
面向典型高性能负载	首先验证 Redis 混部，后续支持 Nginx、MySQL、RPC 和计算型任务	redis-benchmark/memtier、wrk、sys bench 等
相比默认调度器改善不低于 10%	在等条件 A/B 实验中比较吞吐、P99 和 SLO 约束下的有效吞吐	重复实验、置信区间、消融实验
提供完整测试方案和性能数据	建立基线、微基准、负载实验、稳定性测试和 evidence archive	原始数据、配置、内核信息、commit、测试报告

三、需求分析与目标场景

3.1 目标问题

在同一节点上，Redis 服务线程可能需要短响应时间和稳定尾延迟，而 stress-ng、sysbench、批处理或数据处理任务会持续消耗 CPU、缓存和内存带宽。系统需要识别以下差异：

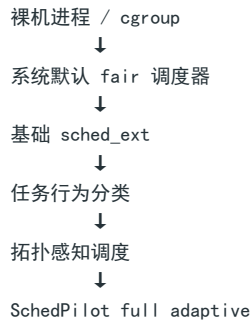
- 延迟敏感、短周期、频繁唤醒的服务线程；

- 长时间运行的 CPU-bound 线程；
- IPC 较低、LLC miss 或内存压力较高的 memory-bound 线程；
- 可延迟但不能无限饥饿的后台任务。

SchedPilot 第一版只把 latency-sensitive、cpu-bound、memory-bound 作为核心分类；cache-sensitive 和 background 作为可选细分标签，避免在第一阶段过度承诺难以区分的类别。

3.2 首验证场景

第一阶段采用以下可归因实验链路：



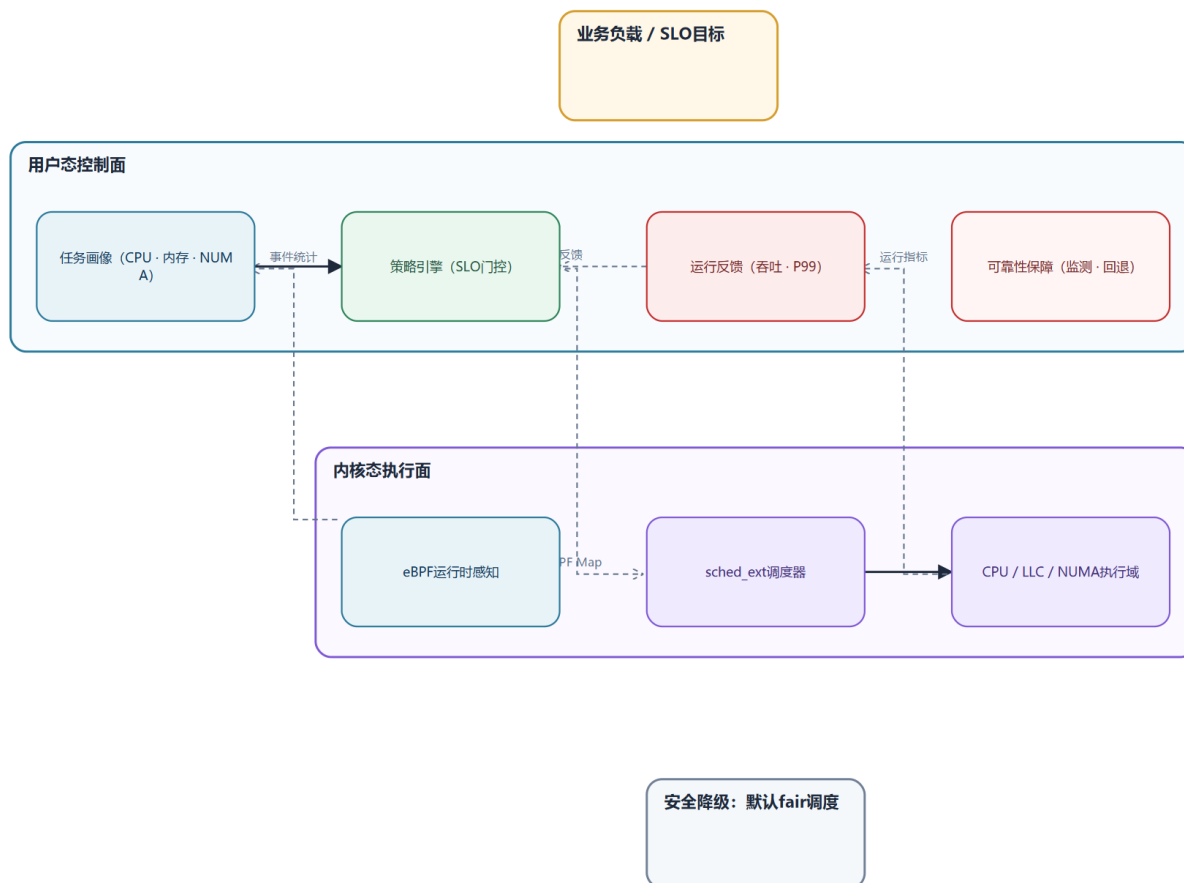
首选负载为 Redis 服务与 CPU/内存密集型干扰任务混部，先不把容器运行时、Kubernetes CPU Manager、Pod QoS 和网络插件引入核心归因链路。性能闭环稳定后，再通过 cgroup 映射扩展至容器和 Kubernetes。

3.3 应用场景

- Redis、MySQL 等在线数据服务的尾延迟保护；
- Nginx、RPC、网关等请求驱动型服务的 SLO 调度；
- 在线服务与离线计算、压测、数据处理任务混部；
- 云主机和边缘节点上的多租户资源隔离；
- Kubernetes 节点级服务等级和资源策略优化。

四、总体解决方案

SchedPilot 由观测、特征、策略、执行、反馈五个部分组成：



SchedPilot 总体技术架构图

4.1 用户态控制面

用户态 Agent 负责：

- 环境能力探测和内核版本识别；
- workload 与 cgroup 发现；
- 特征窗口聚合、分类和置信度计算；
- 策略选择、迟滞、冷却和 SLO gate；
- 更新 BPF map 和 sched_ext 控制参数；
- 记录决策、指标、配置和证据；
- 管理 loader、watchdog、detach 和超时恢复。

4.2 eBPF 观测面

观测指标分为五类：

层级	主要指标	作用
任务级	runtime、wait、wakeup、上下文切换、迁移、sleep/wakeup 周期	判断交互性、抢占和尾延迟风险
CPU 级	cycles、instructions、IPC、run queue、CPU PSI	判断 CPU-bound 与调度拥塞
缓存/内存级	LLC load miss、cache miss、page fault、memory PSI、memory.events	判断 cache/memory 压力
拓扑级	CPU、LLC 域、NUMA node、local/remote memory、迁移距离	判断数据局部性和迁移成本
应用级	Redis 请求率、命令类型、P99、网络吞吐、cgroup	建立服务 SLO 和策略反馈

当目标平台不支持某类硬件计数器时，特征必须标记为 `unavailable`，不能使用零值冒充有效观测；调度器应根据可用特征降级运行。

4.3 特征与分类引擎

每个任务或 `cgroup` 维护滑动窗口特征向量，进行平滑、归一化、置信度计算和超时失效。

- latency-sensitive: 短 runtime、高唤醒频率、等待时间上升或服务 P99 接近 SLO；
- cpu-bound: 长期运行、cycles 占比高、IPC 相对稳定；
- memory-bound: IPC 较低、LLC miss/page fault/memory PSI 较高；
- cache-sensitive: 可选细分标签，要求同时观察 LLC miss 与迁移影响；
- background: 可选细分标签，具备低优先级和饥饿保护。

分类结果携带 `confidence`、`expires_at`、`reason_code` 和特征快照摘要，避免一次异常样本永久改变调度类别。

五、核心创新与技术优势

5.1 面向业务 SLO 的闭环调度

将 Redis P99、请求率、CPU PSI 和任务级等待时间引入调度决策，使调度器从“只看任务”升级为“任务行为与服务目标联合感知”。

5.2 CPU/缓存/内存访问模式联合分类

同时使用调度轨迹、硬件性能计数器、PSI、page fault 和 NUMA locality，区分 CPU-bound 与 memory-bound 任务，避免仅依据进程名或静态优先级做策略判断。

5.3 CPU-LLC-NUMA 多级拓扑感知

根据任务最近运行位置、LLC 域和内存节点选择调度队列，降低不必要的跨 LLC/NUMA 迁移，针对内存密集型负载优化数据局部性。

5.4 用户态快速演进、内核态低开销执行

策略模型和阈值在用户态迭代，BPF map 实现控制面与数据面的低开销通信，sched_ext 在内核态完成关键路径调度。

5.5 可解释、可回滚、可审计

每次分类和策略变更都有原因码、特征摘要、map 更新记录和 DSQ 统计；Agent、loader 或 BPF 异常时通过 watchdog、detach 和超时恢复回到安全调度路径。

5.6 与现有方案的差异

方案	优点	局限	SchedPilot 改进
默认 fair 调度器 (CFS/EE VDF, 以目标内核实现为准)	通用、公平、稳定	缺少业务感知和访问模式驱动的策略	引入 SLO、任务特征和拓扑信息
静态 cgroup 权重/配额	易部署、隔离性好	参数静态, 难以响应负载变化	作为稳定降级, 并由 Agent 动态调整
通用 sched_ext 调度器	可编程、可探索新策略	通常不具备具体业务指标反馈闭环	结合 workload adapter 和 SLO feedback
SchedPilot	观测、分类、拓扑调度、反馈和回滚一体化	依赖内核能力和高质量特征	面向 openEuler 云原生负载进行工程化落地

六、关键技术与调度算法

6.1 多级 DSQ 设计

计划实现：

- 1. per-CPU local DSQ: 处理本地低延迟唤醒；
- 2. per-LLC DSQ: 降低跨 LLC 迁移；
- 3. NUMA memory-bound DSQ: 保持内存局部性；
- 4. latency DSQ: 保护延迟敏感任务；
- 5. throughput DSQ: 承载 CPU-bound 和批处理任务；
- 6. background DSQ: 提供 aging 和 starvation protection；
- 7. shared fallback DSQ: 承载未知任务和降级路径。

6.2 策略动作

- latency-sensitive: 优先本地或 LLC 近端队列，短时间片，必要时受控 kick；

- cpu-bound: 使用公平虚拟时间或吞吐队列, 降低过度抢占;
- memory-bound: 优先 NUMA locality, 限制高代价迁移;
- cache-sensitive: 保持最近运行 CPU/LLC 域, 结合 LLC miss 变化调整;
- background: 低优先级运行, 通过 aging 防止无限饥饿。

6.3 动态门控与稳定性

策略引擎使用激活阈值、恢复阈值、最小保持时间、冷却时间和置信度门槛, 避免分类抖动。策略切换必须满足:

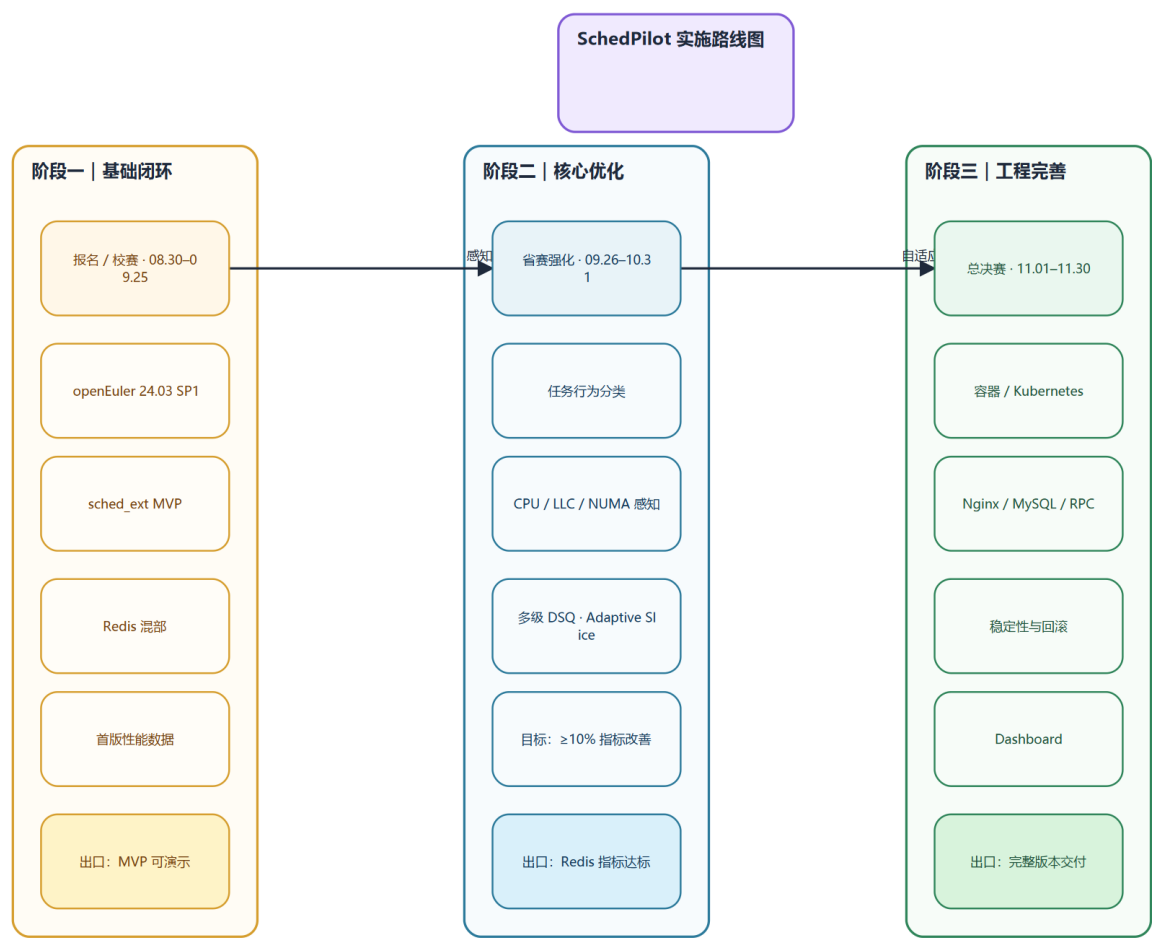
- 单任务和单 cgroup 资源边界;
- 最大连续占用时间;
- background 最大等待时间;
- 调度器异常自动回退;
- 不支持 sched_ext 时降级到 cgroup v2 或系统默认 fair 调度器。

6.4 Loader 与 Watchdog

SchedPilot 不假设 Agent 崩溃后 sched_ext 一定自动恢复, 而是显式实现:

- 独立 loader 负责加载、attach、状态检查和 detach;
- watchdog 检查 Agent 心跳、BPF scheduler 状态、enable sequence 和拒绝计数;
- 心跳超时或异常计数超过阈值时, 停止策略更新并执行安全 detach/降级;
- 重启后通过 map 清理、cgroup 校验和 scheduler identity 检查恢复控制。

七、实施计划



SchedPilot 项目实施路线图

7.1 报名与校赛阶段：2026-08-30 至 2026-09-25 12:00

目标是形成能够回应赛题的最小可演示版本：

- 完成 openEuler 24.03 LTS SP1 目标环境能力探测；
- 完成基础 sched_ext 加载、map、DSQ 和回退闭环；
- 完成裸机/cgroup Redis 混部 workload；
- 完成系统默认 fair、基础 sched_ext、分类调度三组初测；
- 接入 runtime、wakeup、CPU PSI、初步 IPC/LLC/memory 特征；
- 形成第一版性能数据、架构图和方案 PDF。

阶段出口：能够现场证明 scheduler active、任务分类、DSQ dispatch 和安全回退，并拥有第一版可重复实验记录。

7.2 省赛强化阶段：2026-09-26 至 2026-10-31

- 完善 CPU、LLC、NUMA、memory locality 特征；

- 完成 per-CPU/LLC/NUMA 多级 DSQ;
- 完成 adaptive slice、migration cost、aging 和 starvation protection;
- 完成消融实验和至少 20 次重复实验;
- 完成 loader/watchdog、异常恢复和长时间稳定性测试;
- 将策略扩展至容器/cgroup, 并开展 Kubernetes 适配验证。

阶段出口: 在固定环境和真实 ACTIVE policy 下, Redis 核心指标达到 10% 改善, 并完成统计显著性和策略归因证明。

7.3 总决赛完善阶段: 2026-11-01 至 2026-11-30

- 扩展 Nginx、MySQL、RPC 和计算密集型 workload adapter;
- 完成 Dashboard、指标展示和一键演示;
- 完善安装、升级、回滚和内核兼容性矩阵;
- 汇总正式性能报告、工程文档、视频和答辩材料;
- 发布 SchedPilot v1.0 release candidate。

八、测试与性能验证方案

8.1 对照组设计

所有对照必须保持硬件、内核、CPU

绑定、cgroup、数据集、请求模型、干扰负载和实验时长一致。基线名称统一为“系统默认 fair 调度器 (CFS/EEVDF, 以目标内核实际实现为准)”。

至少包含以下组别:

1. default_fair: 系统默认 fair 调度器;
2. basic_sched_ext: 基础 sched_ext, 使用 shared/local 基础路径;
3. classification: 加入 workload classification;
4. topology: 加入 CPU/LLC/NUMA 拓扑感知;
5. full_adaptive: 加入动态时间片、迁移控制、SLO feedback 和回退机制;
6. cgroup_v2: 作为稳定工程后端和辅助对照。

8.2 评价模式

延迟模式: 固定请求速率和负载强度, 比较 P50/P95/P99, 重点观察 Redis P99 是否降低。

吞吐模式: 在满足指定 P99 SLO 的前提下, 比较最大有效吞吐:

Effective Throughput = 满足 P99 ≤ 目标阈值时的最大有效 QPS

避免只报告吞吐上升但 P99 恶化, 或只报告延迟下降但吞吐大幅下降。

8.3 负载矩阵

- Redis GET、SET、INCR、PING;

- Redis 服务 + stress-ng/sysbench CPU 干扰；
- Redis 服务 + 内存带宽/随机访存干扰；
- 安静环境、CPU 压力环境、内存压力环境；
- 裸机进程/cgroup 环境；
- 容器/Kubernetes 环境作为扩展验证。

8.4 数据与统计要求

每个核心场景至少进行 20 次有效重复，报告：

- 平均值、中位数、标准差；
- P50/P95/P99；
- 吞吐和 Effective Throughput；
- CPU 利用率、上下文切换、迁移、LLC miss、memory PSI；
- 95% 置信区间和异常样本处理规则；
- active policy、类别命中、DSQ dispatch、运行计数；
- 内核版本、硬件拓扑、配置、Git commit 和 artifact ID。

8.5 性能验收门槛

在固定硬件、固定内核和固定 workload 下，full_adaptive 相比 default_fair 至少满足一项：

- 吞吐提升不低于 10%；
- P99 延迟降低不低于 10%；
- 主要指标置信区间不跨越 0；
- 不出现超过 5% 的主要回归；
- 策略命中和 DSQ 统计能够证明收益来自 SchedPilot 的 ACTIVE adaptive policy。

调度器额外开销只设为控制目标，不在没有数据支撑前写死为硬性结果：目标是通过采样率、per-CPU map 和低频聚合将控制面与观测面开销控制在可接受范围内，最终以同条件实测数据为准。

九、可行性、风险与安全机制

风险	影响	应对措施
SP1 目标内核与上游 sched_ext 接口存在差异	编译、加载或语义不一致	以能力探测为入口，隔离 kernel adapter，建立 openEuler 与上游内核矩阵
硬件计数器不可用	访问模式特征不完整	特征显式标记 unavailable，采用可用子集并报告影响
全局 DSQ 竞争	吞吐下降、延迟抖动	使用 per-CPU/LLC DSQ，限制跨域迁移
分类抖动	策略频繁切换	滑动窗口、置信度、迟滞、冷却和最小保持时间
Agent 或 loader 异常	调度控制失效	watchdog、心跳超时、显式 detach、cgroup v2/fair fallback
性能收益归因不清	评审无法确认创新点	基础 sched_ext、分类、拓扑、full adaptive 消融实验
历史结果与最终结果混杂	证据可信度下降	每组结果绑定 commit、配置、内核、原始数据和唯一 artifact ID
sched_ext ABI 变化	后续维护成本增加	统一接口层、compile/load smoke test 和版本适配文档

安全边界包括 PID/TGID 校验、cgroup 归属校验、目标路径校验、权限检查、非目标任务保护和异常状态下的最小权限回退。

十、产业价值与推广方式

10.1 产业价值

- 提升国产操作系统在云原生、高性能服务和边缘计算场景中的资源调度能力；
- 在不修改业务代码的情况下改善服务尾延迟和节点资源利用率；
- 降低服务 SLO 与底层 CPU 调度之间的割裂；
- 为多租户节点提供可解释、可观测、可回滚的调度策略；
- 形成适配 openEuler 的 eBPF/sched_ext 调度技术样板。

10.2 推广路径

1. 以 Redis 混部作为可复现的性能示范；
2. 通过 workload adapter 扩展到 Nginx、MySQL 和 RPC；
3. 通过 cgroup 和 Node Agent 接入 Kubernetes；
4. 通过 Dashboard、策略模板和指标接口支持运维平台集成；
5. 形成 openEuler 节点级调度优化组件和行业场景方案。

十一、项目成果与交付物

- SchedPilot Agent 用户态控制程序；
- eBPF workload observer 和性能特征采集程序；
- sched_ext BPF scheduler、loader 和 watchdog；
- Redis、Nginx、MySQL、通用 cgroup workload adapter；
- 安装、启动、停止、回滚和环境检查脚本；
- 单元、集成、稳定性和故障恢复测试；
- Redis 混部性能数据、消融实验和原始证据；
- 架构设计、部署手册、实验手册、故障排查和兼容性文档；
- 正式 PDF 解决方案、演示视频和答辩材料。

十二、团队分工建议

角色	主要职责
系统架构与项目负责人	需求拆解、总体架构、版本和交付管理
eBPF/sched_ext 负责人	BPF observer、DSQ、loader、watchdog 和内核适配
性能与算法负责人	特征工程、分类、拓扑策略、消融实验和统计分析
云原生与测试负责人	cgroup/Kubernetes、workload、自动化测试和复现环境
文档与展示负责人	方案 PDF、架构图、Dashboard、视频和答辩材料

十三、v1.0 验收标准

SchedPilot v1.0 必须同时满足：

1. 在 openEuler 24.03 LTS SP1+ 目标环境完成能力探测、安装和运行；
2. 明确兼容上游 Linux 6.12+ sched_ext 接口，并记录差异；
3. sched_ext ACTIVE 路径可由状态、map、类别命中和 DSQ 统计证明；
4. 至少稳定区分 latency-sensitive、cpu-bound、memory-bound 三类任务；
5. 自适应策略实际使用 CPU/缓存/内存访问特征中的至少两类；
6. Redis 混部场景相比系统默认 fair 调度器吞吐或 P99 改善至少 10%；
7. 核心结果至少 20 次重复，置信区间和异常处理规则完整；
8. Agent、loader、BPF scheduler 或内核能力异常时能够安全回退；
9. 其他开发者可依据文档完成部署、测试和结果复现。

十四、正式提交材料规划

本计划的正式交付文件为：

SchedPilot 项目计划书.pdf

正式提交版 PDF 应包含：

- 封面：项目名称、命题名称、产业赛道、企业命题组、参赛学校、负责人、指导教师和日期；
- 目录；
- 彩色系统架构图；
- 赛题响应矩阵；
- 关键技术和创新点；
- 消融实验和正式性能数据；
- 部署、回滚和兼容性说明；
- 产业价值、竞争分析和项目成果。

本计划采用可读的彩色系统架构图呈现总体方案，文字章节补充模块职责、实施步骤和验收要求。